**PROGRAM 11 – MAP COLORING USING CSP**

**AIM**

To write a Python program to solve the Map Coloring problem using Constraint Satisfaction Problem (CSP) and Backtracking.

**ALGORITHM**

1. Start the program.
2. Define the map as a graph with regions and their neighbouring regions.
3. Define a set of available colours.
4. Assign a colour to each region.
5. Check whether neighbouring regions have different colours.
6. If the colour satisfies the constraint, continue with the next region.
7. If no colour is valid, backtrack.
8. Repeat until all regions are assigned colours.
9. Display the final colour assignment.

**CODE**

```
regions=["A","B","C","D"]
graph={"A":["B","C"],"B":["A","C","D"],"C":["A","B","D"],"D":["B","C"]}
colors=["Red","Green","Blue"]
assignment={}
def is_safe(region,color):
    for neighbor in graph[region]:
        if assignment.get(neighbor)==color:
            return False
    return True
def solve(index):
    if index==len(regions):
        return True
    region=regions[index]
    for color in colors:
        if is_safe(region,color):
            assignment[region]=color
            if solve(index+1):
                return True
```

```
            del assignment[region]
    return False
if solve(0):
    print("Map Coloring Solution:")
    for region,color in assignment.items():
        print(region,":",color)
else:
    print("No solution found")
```

**INPUT**

```
regions=["A","B","C","D"]
graph={"A":["B","C"],"B":["A","C","D"],"C":["A","B","D"],"D":["B","C"]}
colors=["Red","Green","Blue"]
```

**OUTPUT**

```
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red
```

**RESULT**

Thus, the Map Coloring problem was successfully implemented using CSP and Backtracking in Python.